

Evaluation

What is measured, on what, and what would change the conclusion.

What `--replay` does and does not re-derive

```
uv run brandbot eval --replay
```

Reads two things: the hand-written labels in `data/gold/golden.jsonl`, and the recorded runs in `artifacts/runs/`. Recomputes every headline figure from them. No key, no network, no local model server.

It **does** re-derive: all intent metrics and their intervals, coverage, risk, human touches, the paired comparisons between systems, per-intent breakdowns, and judge-human agreement.

It **does not** re-derive: the model's predictions or the judge's verdicts. Those were produced by `brandbot run`, which needs both API keys and a local embedding server, and they are committed as a record. Re-running them would change the numbers slightly even at temperature zero, so the record is what is replayed.

This is a deliberate trade. Replaying the recorded verdicts means a reviewer checks the arithmetic and the reasoning, not the sampling luck of one particular afternoon. The full pipeline is reproducible with `brandbot run`, and every model response is cached on disk under `artifacts/cache/`, keyed by model and the exact rendered prompt, so re-running it hits the cache rather than the provider.

Metrics

Intent

Four numbers, reported together because they disagree and picking the flattering one is the specific failure this project exists to avoid.

Macro-F1 weights every intent equally, including ones that arrive twice a month. This is the headline for "how well does it do per intent", and it is the number the golden set is stratified to support.

Plain accuracy is the number to distrust. On a set with an uneven intent mix it mostly reports how common the commonest intent is.

Prior-weighted accuracy reweights per-intent recall by the intent mix actually observed in the uniformly drawn part of the golden set. It answers a different question from macro-F1: what fraction of tomorrow's unsorted traffic would land correctly. It is usually the more flattering of the two, which is exactly why it is never quoted alone.

Same-next-step accuracy asks whether the mistake changed anything. The ten intents collapse into five handling groups, defined by what a support team would actually do: troubleshoot a fault, answer a catalogue question, explain a policy, send it to a human, or just acknowledge it. Confusing `app_bug` with

`service_outage` costs nothing, they get the same treatment. Confusing `billing_refund` with `autoplay_error` auto-replies to a money dispute. Macro-F1 charges the same price for both.

All intervals are 95%, from 2,000 bootstrap resamples of the examples. Comparisons between systems are **paired**: both systems are scored on the same resampled examples, so the shared difficulty of those examples cancels instead of inflating the spread. Comparing two independent intervals would be the wrong test and would usually be the more flattering one.

Escalation

Escalation is a choice about how much to answer, so it is measured as a pair and never as a single number.

Coverage is the share of messages answered without a human. **Risk** is the share of those answers a support lead would have stopped.

Either one alone is trivially gameable: escalate everything and risk is zero; send everything and coverage is 100%. The pair is the result.

Human touches per 100 messages combines them under an explicit cost. One escalation is one touch. One reply that should not have gone out costs five, covering the apology, the re-handling and the damage. Five is a stated placeholder, not a measured figure for this brand, so the report sweeps it and shows where the ranking changes.

The content rules in `pipeline/policy.py` run before any confidence is consulted. A billing dispute goes to a person whether the draft looked good or not, because the cost of being wrong is not symmetric and no confidence score prices it correctly.

Rubric

The judge answers five questions about one draft reply, given the customer's message and the precedents that were retrieved. Yes or no, never a scale.

Question	
grounded	Is every fact, fix, link, timeline and policy in it supported by the past replies? A plausible invention is not grounded.
addresses	Does it answer what this customer asked, rather than a neighbouring question?
safe	Does it avoid promising anything the brand would be held to and the past replies do not already promise?
tone	Does it read like this brand's own support account?
send	Would you let this go out unedited, to this customer, right now?

`send` is the headline. The four dimensions exist to say why something failed and are deliberately not summed into it. Where `send` disagrees with all four dimensions passing, the disagreement is reported rather than resolved: it is the cheapest evidence available about how steady the judge is.

A scale from one to five was rejected. Nobody can say what separates a 3 from a 4, two raters will not draw that line in the same place, and a rubric that cannot be applied consistently produces an agreement statistic that flatters itself. "Would you send it" is a decision a support lead makes dozens of times a day.

Asking the customer a clarifying question passes. Some messages rest on a premise nobody can verify from one tweet, and a reply that asserts through the uncertainty is worse than one that asks.

Judge validation

An LLM judge is an instrument, and an uncalibrated instrument is decoration. A human graded a blind sample against the same rubric, and agreement is reported as Cohen's kappa, which asks whether two raters agree more than chance alone would explain. That matters here because the labels are skewed: on a rubric where most replies pass, two raters who both say "send" every time agree most of the time while carrying no information at all. Raw percent agreement is reported alongside it, and the gap between the two is itself evidence about how skewed the label is.

Three rules protect the comparison.

The sample is drawn at random within each system, never by verdict. Choosing replies the judge was confident about, or ones it failed, selects on the thing being measured.

The human sees the message, the reply and the precedents. Not which system wrote it, and not what the judge said.

Systems are sampled in roughly equal numbers rather than in proportion, because the trivial baseline sends one identical line to every message and a proportional draw would spend most of the human's attention re-reading the same sentence. Agreement is reported per system as well as overall: a judge can be reliable on obvious failures and useless on the close calls, and only the close calls decide anything.

Cross-judging

A second model from a third family re-grades a sample. An LLM judge rewards replies that look like something it would have written, and using a different family from the systems under test removes the worst of that but not all of it. The gap between the two judges is reported.